

Spark-анализ сессий КонсультантПлюс

Выполнил: Васюков Александр

Краткое описание

Реализован проект, который читает многострочные логи сессий КонсультантПлюс, выделяет события поиска и открытия документов, валидирует связь открытия с поиском и считает две требуемые метрики.

1. Количество раз, когда в карточке производили поиск документа с идентификатором ACC_45616
2. Количество открытий каждого документа, найденного через быстрый поиск за каждый день. Формат вывода результата можно выбрать самостоятельно.

Какие данные есть в логах

В data лежит 10000 файлов без расширений. Каждый файл — одна пользовательская сессия. Сессия начинается с SESSION_START и заканчивается SESSION_END; внутри неё встречаются поиски и открытия документов.

Важная особенность данных: это не табличный CSV и не JSON. Одно логическое событие часто занимает несколько строк. Поэтому простое прямое решение вида «прочитать строку, split по пробелу, сразу посчитать» не подходит: строка QS сама по себе не содержит список документов, а строка CARD_SEARCH_START не содержит ни параметры, ни результат поиска.

На какие файлы полезно смотреть при проверке формата:

- data/1069 — простой пример QS: строка быстрого поиска, затем строка результатов и несколько DOC_OPEN.

```
data > ≡ 1069
1  SESSION_START 23.06.2020_03:53:29
2  QS 23.06.2020_03:55:06 {0000000 000000000 0 153}
3  276347703 LAW_71050 LAW_147539 LAW_70881 EPB_579144 LAW_
4  DOC_OPEN 23.06.2020_03:56:47 276347703 LAW_295605
5  DOC_OPEN 23.06.2020_03:57:46 276347703 LAW_295605
6  DOC_OPEN 23.06.2020_03:57:54 276347703 LAW_70881
7  DOC_OPEN 23.06.2020_03:58:52 276347703 LAW_71050
8  DOC_OPEN 23.06.2020_03:59:47 276347703 LAW_295605
9  DOC_OPEN 23.06.2020_04:01:19 276347703 LAW_147539
10 QS 23.06.2020_04:03:01 {1799}
11 406151988 CJI_118747 CMB_18794 PBI_269188 LAW_367880 LAW_
12 DOC_OPEN 23.06.2020_04:03:25 406151988 CJI_121359
13 DOC_OPEN 23.06.2020_04:04:32 406151988 LAW_347465
14 DOC_OPEN 23.06.2020_04:05:51 406151988 LAW_6176
15 DOC_OPEN 23.06.2020_04:07:42 406151988 LAW_367875
16 DOC_OPEN 23.06.2020_04:08:38 406151988 LAW_6176
17 DOC_OPEN 23.06.2020_04:10:26 406151988 LAW_251027
18 DOC_OPEN 23.06.2020_04:11:26 276347703 LAW_71050
19 SESSION_END 23.06.2020_04:12:04
```

- data/2618 — пример карточного поиска с параметром \$134, CARD_SEARCH_END и строкой результатов.

```
data > 2618
1  SESSION_START 01.12.2020_09:02:39
2  QS 01.12.2020_09:03:18 {00000 12 00000013 00 00 23.11.2
3  27812326 CJI_91998 LAW_330075 PBI_146414 PBI_214680 PBI
4  DOC_OPEN 01.12.2020_09:04:16 27812326 QSB0_8192
5  DOC_OPEN 01.12.2020_09:04:53 27812326 PBI_200367
6  DOC_OPEN 01.12.2020_09:05:44 27812326 PBI_202882
7  DOC_OPEN 01.12.2020_09:07:35 27812326 PBI_179512
8  DOC_OPEN 01.12.2020_09:09:34 27812326 CMB_17702
9  DOC_OPEN 01.12.2020_09:10:29 27812326 ACC_45616
10 QS 01.12.2020_09:11:30 {0 0000000000 00 0000 0 000000000
11 14499388 PBI_136770 PBI_62370 ACC_45616 PBI_147270 PBI_
12 DOC_OPEN 01.12.2020_09:12:49 14499388 PBI_204020
13 DOC_OPEN 01.12.2020_09:13:27 14499388 PBI_147270
14 QS 01.12.2020_09:14:28 {00000000 00 0000000000}
15 25037489 PSP_3 PSP_16 PBI_236893 PBI_262946 LAW_323656
16 DOC_OPEN 01.12.2020_09:16:10 25037489 LAW_146628
17 DOC_OPEN 01.12.2020_09:17:08 25037489 PBI_236893
18 DOC_OPEN 01.12.2020_09:19:03 25037489 CMB_18767
19 DOC_OPEN 01.12.2020_09:19:54 14499388 PBI_241757
20 CARD_SEARCH_START 01.12.2020_09:20:36
21 $134 199-0
22 $0 ARB_205613
23 CARD_SEARCH_END
24 616630221 ARB_205613
25 DOC_OPEN 01.12.2020_09:21:59 616630221 ARB_205613
26 DOC_OPEN 01.12.2020_09:22:36 616630221 ARB_205613
27 SESSION_END 01.12.2020_09:25:00
```

- data/3701 — пример даты в формате Fri,_26_Jun_2020_16:24:28_+0300 и карточного поиска, где среди результатов есть ACC_45616.

```
data > 3701
1  SESSION_START 26.06.2020_16:23:23
2  CARD_SEARCH_START Fri,_26_Jun_2020_16:24:28_+0300
3  $134 0000000000000 0 000000000
4  CARD_SEARCH_END
5  1823433 DOF_61914 DOF_86333 DOF_72712 DOF_53390 DOF_389
6  DOC_OPEN 26.06.2020_16:25:19 1823433 DOF_59163
7  DOC_OPEN 26.06.2020_16:25:56 1823433 DOF_53606
8  DOC_OPEN 26.06.2020_16:26:37 1823433 DOF_99850
9  DOC_OPEN 26.06.2020_16:28:26 1823433 DOF_84803
10 DOC_OPEN 26.06.2020_16:29:02 1823433 LAW_169197
11 DOC_OPEN 26.06.2020_16:30:33 1823433 DOF_38960
12 QS Fri,_26_Jun_2020_16:31:50_+0300 {000000 0000000000 00
13 37785281 PBI_263130 PBI_233956 PBI_237138 PBI_261568 PB
14 DOC_OPEN 26.06.2020_16:33:38 37785281 PBI_119412
15 DOC_OPEN 26.06.2020_16:33:41 37785281 PBI_241647
16 DOC_OPEN 26.06.2020_16:33:48 37785281 PBI_222240
17 DOC_OPEN 26.06.2020_16:34:08 37785281 PBI_241647
18 DOC_OPEN 26.06.2020_16:35:48 37785281 PBI_233956
19 DOC_OPEN 26.06.2020_16:37:27 37785281 PBI_119412
20 SESSION_END 26.06.2020_16:38:28
```

- data/7158 — пример DOC_OPEN без timestamp: DOC_OPEN 5181406 PAP_1393.

```
data > ≡ 7158
1  SESSION_START 08.08.2020_04:16:47
2  QS 08.08.2020_04:17:59 {000000 00000000000000 00 0000000000
3  5181406 PAP_4855 PAP_25526 PAP_4692 PAP_25528 PAP_25354
4  DOC_OPEN 5181406 PAP_1393
5  DOC_OPEN 5181406 PAP_34811
6  DOC_OPEN 5181406 PAP_23002
7  DOC_OPEN 5181406 PAP_43720
8  DOC_OPEN 5181406 PAP_90090
9  DOC_OPEN 5181406 PBI_199791
10 QS 08.08.2020_04:21:23 {00000000 0000000000000000 0000000000
11 -1931766530 RLAW077_117380 ACC_45616 ACC_45615
12 DOC_OPEN -1931766530 ACC_45615
13 DOC_OPEN -1931766530 ACC_45616
14 SESSION_END 08.08.2020_04:23:54
```

Данные меняются по времени и формату:

- обычная дата: 23.06.2020_03:55:06;
- дата с днём недели и timezone: Fri,_26_Jun_2020_16:24:28_+0300;
- часть DOC_OPEN записана без даты;
- в карточном поиске количество параметров \$... различается;
- длина списка найденных документов различается от одного документа до десятков.

Что такое QS и CARD_SEARCH_START

QS (quick search) — быстрый поиск через поисковую строку. У него есть текст запроса и строка результатов. Формат в файле выглядит так:

```
QS 23.06.2020_03:55:06 {приказ минтруда № 153}
276347703 LAW_71050 LAW_147539 LAW_70881 ...
```

Здесь 276347703 — searchId, а всё после него — документы, найденные быстрым поиском. Позже DOC_OPEN ссылается на этот searchId:

```
DOC_OPEN 23.06.2020_03:56:47 276347703 LAW_295605
```

CARD_SEARCH_START — начало карточного поиска. Это поиск не одной строкой, а набором параметров. Между CARD_SEARCH_START и CARD_SEARCH_END идут строки вида \$<id параметра> <значение>, затем отдельная строка результата:

```
CARD_SEARCH_START 01.12.2020_09:20:36
$134 199-и
$0 ARB_205613
CARD_SEARCH_END
616630221 ARB_205613
```

Уникальность этих событий в том, что Q5 и CARD_SEARCH_START нельзя корректно разобрать независимо по одной строке. Нужно видеть соседние строки одного файла-сессии.

Почему прямое решение не подходит

Рассматривались три варианта:

1. **Построчный парсинг через `textFile`.** Простой, но ломается на многострочных событиях: строка Q5 не содержит найденных документов, а карточка поиска вообще является блоком.
2. **Предварительно склеить логи в JSON/CSV отдельным скриптом.** Удобно для анализа, но появляется лишний слой ETL, который надо отдельно проверять.
3. **Читать каждый файл целиком через `Spark wholeTextFiles`.** Этот вариант выбран: Spark параллелит файлы, а парсер внутри одного файла видит контекст соседних строк.

Именно третий вариант лучше соответствует задаче: он не теряет структуру сессии и не требует промежуточного формата.

Что делает `wholeTextFiles`

`wholeTextFiles(inputPath)` — метод `SparkContext`, который читает не отдельные строки, а целые файлы. На выходе получается RDD пары:

(путь_к_файлу, полный_текст_файла)

В этом задании такой подход удобен, потому что один файл равен одной сессии, а события внутри сессии связаны между строками. После `wholeTextFiles` парсер получает полный текст файла, проходит по строкам и может:

- после Q5 прочитать следующую строку результатов
- после CARD_SEARCH_START найти соответствующий CARD_SEARCH_END

Технологии

- Scala.
- Spark

Почему Scala/Spark:

- Scala требовалось по условию задачи
- Spark имеет распределённый подход и масштабируется на большой объём
- строгий контракт с типизированными `case class`
- парсер отделён от Spark job

Альтернативы:

- Python / Go
- PySpark

Модель данных в коде

В проекте есть три основные модели.

SearchEvent:

- searchId — нужен для связи с DOC_OPEN
- searchType — отличает QS от CARD, чтобы не смешать метрики
- eventDate — дата поиска, нужен для восстановления даты открытия
- documents — список найденных документов, по нему проверяется ACC_45616
- sourceFile — часть ключа связи, потому что searchId безопасен внутри файла-сессии, но не обязан быть глобально уникальным
- lineNumber — помогает найти исходное место события в логе

DocOpen:

- searchId — ссылка на поиск
- documentId — открытый документ
- eventDate — дата открытия, если она есть в строке
- sourceFile — используется вместе с searchId
- lineNumber — исходное место в логе

ParseWarning:

- sourceFile, lineNumber, message — минимальный набор, чтобы не падать на битой записи, но иметь возможность найти и объяснить проблему.

Восстановление даты DOC_OPEN

В части файлов DOC_OPEN записан без timestamp. Например, в data/7158:

```
QS 08.08.2020_04:17:59 { ... }
5181406 PAP_4855 PAP_25526 ... PAP_1393 ...
DOC_OPEN 5181406 PAP_1393
```

В этой строке открытия нет даты, но есть searchId = 5181406. Он ссылается на предыдущий QS, где дата есть: 08.08.2020. Поэтому для такой записи используется дата связанного быстрого поиска.

Это решение лучше пропуска строки, потому что открытие валидно связано с поиском и документ действительно был среди найденных. При этом дата восстанавливается только для метрики быстрого поиска; карточные открытия во вторую метрику не попадают.

Архитектура обработки

data/*	->	wholeTextFiles	->	LogParser	->	SearchEvent / DocOpen
--------	----	----------------	----	-----------	----	-----------------------

Таблица 1. Чтение и нормализация логов

SearchEvent + DocOpen	->	Spark SQL join + validation	->	CSV results
-----------------------	----	-----------------------------	----	-------------

Таблица 2. Расчёт метрик и запись результатов

Последовательность работы:

1. Spark читает файлы как (path, content).
2. LogParser.parseFile выделяет поиски, открытия и предупреждения.
3. Первая метрика фильтрует CARD-поиски по наличию ACC_45616.
4. Вторая метрика берёт только QS, связывает с DOC_OPEN по (sourceFile, searchId), проверяет наличие документа в результатах QS и группирует по дню и документу.
5. Результаты пишутся в results/.

Логика расчёта метрик

Метрика 1

Берём только SearchEvent, где:

```
searchType == "CARD" && documents contains "ACC_45616"
```

Результат:

document_id	card_search_count
ACC_45616	479

Таблица 3. Количество карточных поисков ACC_45616

Метрика 2

Условие попадания DOC_OPEN в результат:

- есть связанный QS;
- связь строится по (sourceFile, searchId);
- documentId был в списке найденных документов этого QS;
- дата есть в DOC_OPEN или восстанавливается из QS.

Полный результат записан в формате date, document_id, open_count. Это удобный формат: он не создаёт много колонок и легко читается Spark, SQL, Pandas или Excel.

Пример первых строк полного результата:

date	document_id	open_count
2020-01-01	ACC_45614	3
2020-01-01	ACC_45615	7
2020-01-01	ACC_45616	1

Таблица 4. Фрагмент qs_doc_opens_by_day.csv

В данных 54405 уникальных документов. Если сделать таблицу вида день, документ_1, документ_2, ... для всех документов, получится файл с 54406 колонками. Такой файл технически возможен, но почти нечитабелен: его неудобно открывать, сравнивать и вставлять в отчёт. Поэтому результат записан в виде таблицы по 30 самым часто открываемым документам:

Обработка ошибок

Парсер не бросает исключение на неполной записи, а добавляет ParseWarning. Это важно для логов: одна испорченная строка не должна ломать весь расчёт.

Проверяются случаи:

- QS без строки результатов;
- CARD_SEARCH_START без CARD_SEARCH_END;
- CARD_SEARCH_END без строки результатов;
- DOC_OPEN с недостаточным числом полей;
- дата в неизвестном формате.

В полном датасете после запуска ошибок не было найдено.

```
parse_warnings_count=0
```

Итог реализации

Выполнены пункты задачи:

- Spark-задача реализована на Scala
- изучены данные и описаны особенности формата
- посчитана метрика карточного поиска ACC_45616
- посчитана дневная статистика открытий документов, найденных через QS

Фактический вывод полного spark-submit:

```
card_acc_45616_count=479
qs_doc_opens_by_day_rows=75516
qs_doc_opens_by_day_wide_top_documents=30
parse_warnings_count=0
```

Как запустить

Обычный запуск через sbt:

```
cd project
sbt test
sbt assembly
spark-submit \
  --class ru.hse.datapipelines.consultant.ConsultantLogJob \
  --master 'local[*]' \
  target/scala-3.8.3/consultant-spark-project.jar \
  data results
```

Если используется jar из target/:

```
cd project
spark-submit \
  --class ru.hse.datapipelines.consultant.ConsultantLogJob \
  --master 'local[*]' \
  target/consultant-spark-project.jar \
  data results
```

Вывод

Решение построено вокруг реального формата логов. `wholeTextFiles` сохраняет контекст сессии, `case class` явно фиксируют смысл данных, а отдельный файл предупреждений делает обработку устойчивой к ошибкам в логах. Выбранный подход остаётся простым, но покрывает главные сложности: многострочные события, два формата дат, неполные `DOC_OPEN` и разделение быстрого и карточного поиска.